

GPU 記憶體與資料搬遷讀書會 · S1-S5 全系列合輯

速度的故事： 平行度 × 資料搬運



roofline → 共同演化

從 roofline 與記憶體階層出發，看懂訓練 / 推論的瓶頸、資料搬遷的每道關卡，
最後拉高視角：模型設計與計算機結構如何互相塑造（CNN → Transformer → 混合架構）。

聽眾：data science 背景 · 原 S1-S5 五場內容重編去重

01

開場謎題：最貴的算力，為什麼用不到？

謎題

< 5%

同一張 H100，batch=1 的 LLM 解碼，tensor core 利用率常不到 5%——買了全世界最貴的算力，卻幾乎用不到。為什麼？

答案不在「算力不夠」，而在「資料搬不夠快 + 平行度不夠」——整份合輯就在拆這件事。

02 路線圖：五個篇章

1 機器

GPU 是台什麼樣的機器： SM / warp / Amdahl / 記憶體階層

2 一把尺

算術強度與 roofline： compute-bound 還是 memory-bound？

3 模型上機

訓練 vs 推論： prefill / decode / KV cache / batch （Transformer 推論）

4 資料搬遷

把搬運藏在運算後面： prefetch / overlap （壓軸）

5 共同演化

拉高視角：模型設計 $\Leftrightarrow$ 計算機結構（CNN $\rightarrow$ Transformer $\rightarrow$ 混合）

主線：先認識機器 $\rightarrow$ 拿到尺 $\rightarrow$ 量模型 $\rightarrow$ 修搬運 $\rightarrow$ 回頭看懂模型為什麼長這樣。



互動環節

講者：切出去打開互動式地圖，從 Cluster 一路切到 SM、再到 CUDA / Tensor core

interactive/gpu_map.html



點發亮的元件往內切 · Esc 回上層 · 1-5 直接跳層 · 講完按 5 停在運算單元，回來接下一頁

03 warp 與 SIMT：用大量 thread 藏延遲

32 threads = 1 warp，一起執行同一指令（SIMT）。一個 SM 同時駐留很多 warp。



時間 → 一個 warp 在等 HBM（數百 cycle）時，排程器切到別的 warp 繼續算 → SM 始終有人在算。橘 = 在算 深 = 等 HBM

藏延遲的前提：有夠多 warp、且頻寬餵得上——「大量平行」不是 GPU 的加分項，是它存在的前提。

04

餵飽一張卡要多少平行度？ Amdahl 給天花板

H100 SXM 的「寬」



132 SM $\times$ 128 lane $\approx$ 16,896 條 lane + 528 tensor core

每條 lane 還要數十個 warp 待命藏延遲

→ 同時要有 10^5 量級的 thread 在飛

Amdahl's Law

$$\text{加速上限} = 1 / ((1-p) + p/N)$$

p = 可平行比例, N = lane 數

$p = 95\%$ 、 $N \rightarrow \infty$: 最多也只有 20x

N 夠大之後, 瓶頸只剩序列的 $(1-p)$

模型裡任何「序列相依」的部分, 都是 p 的天花板——這句話 Part 3、Part 5 都會用到。

05

記憶體階層：每遠一站，慢一個數量級

層級	約略頻寬	誰管理	備註
Register	數十 TB/s	編譯器	晶片內最快
Shared memory / L1	數十 TB/s	程式可控	tiling 的槓桿（Part 2）
L2 cache	數 ~ 數十 TB/s	硬體	全 SM 共用
HBM（GPU 全域）	2 ~ 4.8 TB/s	程式配置	A100~2 / H100~3.35 / H200~4.8
NVLink（GPU↔GPU）	~900 GB/s	—	比 PCIe 快一個量級
PCIe（Host↔Device）	Gen4 ~32 / Gen5 ~64 GB/s	—	最常被跨越的瓶頸
CPU DRAM（DDR5）	~50 ~ 100+ GB/s	OS	主機記憶體
NVMe SSD	~3 ~ 7 GB/s	OS	資料集 / 權重來源

HBM 是 TB/s、PCIe 是 GB/s——差約 100×；瓶頸＝資料必經的「最慢那段路」（常是 PCIe）。能程式控制的只有 shared memory 與 register。

06

為什麼越快的記憶體越小？看每 GB 價格（2026 初）

↑ 越貴 · 越快 · 越小

片上 SRAM

\$5,000/GB 起

KB ~ MB · 數十 TB/s

HBM3E

\$13–17/GB

數十 ~ 141 GB · ~3–5 TB/s

一般 DRAM (DDR5)

\$2–3/GB

TB 級 · ~100 GB/s

↓ 越便宜 · 越慢 · 越大

價差有多誇張

SRAM $\approx$ HBM 的 $\sim 300\text{--}2000\times$

HBM $\approx$ DDR5 的

$\sim 5\text{--}8\times$

所以呢

- 晶片內貴到只放得起 KB~MB $\rightarrow$ SRAM 註定很小
- KV cache (GB) 塞不進 SRAM $\rightarrow$ 只能住 HBM、decode 每步串流 (Part 3)
- Groq 全 SRAM $\rightarrow$ 一顆只有 230MB、貴 $\rightarrow$ 要很多顆 (Part 5)

頻寬階層（上一頁）與價格階層是同一回事：越快越貴越小 $\rightarrow$ 「容量 / 頻寬 / 成本」三角逼出整個記憶體階層。

價格為 2026 初概略合約價（HBM3E 2025 H1 高峰曾 \$17–20）；SRAM 為片上等效估計、非市售模組。

07

方案速覽與選型心法

方案	記憶體	容量	頻寬	定位
RTX 4090	GDDR6X	24 GB	~1 TB/s	開發 / 本機
A100 80GB	HBM2e	80 GB	~2 TB/s	訓練 / 推論
H100 SXM	HBM3	80 GB	~3.35 TB/s	主力
H200	HBM3e	141 GB	~4.8 TB/s	LLM 推論
Apple M 系列	統一 LPDDR	128—192 GB+	~0.4—0.8 TB/s	本機大模型
Grace Hopper	HBM3+LPDDR5X	96 + ~480 GB	HBM ~4 TB/s	超大模型

決策：裝得下 HBM → 純 GPU；裝不下但搬得動 → offload/UVM/GDS + overlap；要超大容量 → 統一記憶體。

memory-bound 推論看「頻寬 + 容量」，不是峰值 FLOPS。沒有最好的記憶體，只有最 match 工作集的。

08

兩種「慢」：你被誰卡住？

Compute-bound

- 瓶頸：峰值 FLOPS（算得不夠快）
- 症狀：算力利用率高、加大 batch 也快不了
- 典型：大矩陣乘法、模型訓練

Memory-bound

- 瓶頸：記憶體頻寬（資料搬不夠快）
- 症狀：算力利用率低、加大 batch 吞吐會升
- 典型：自迴歸解碼、element-wise 運算

怎麼判斷在哪一邊？用一把尺：算術強度（下一頁）。

09 一把尺：算術強度 (Arithmetic Intensity)

$$\text{AI} = \frac{\text{完成運算的 FLOPs}}{\text{需搬動的 Bytes}}$$

單位：FLOPs / Byte —— 每搬 1 個位元組，能換到幾次浮點運算

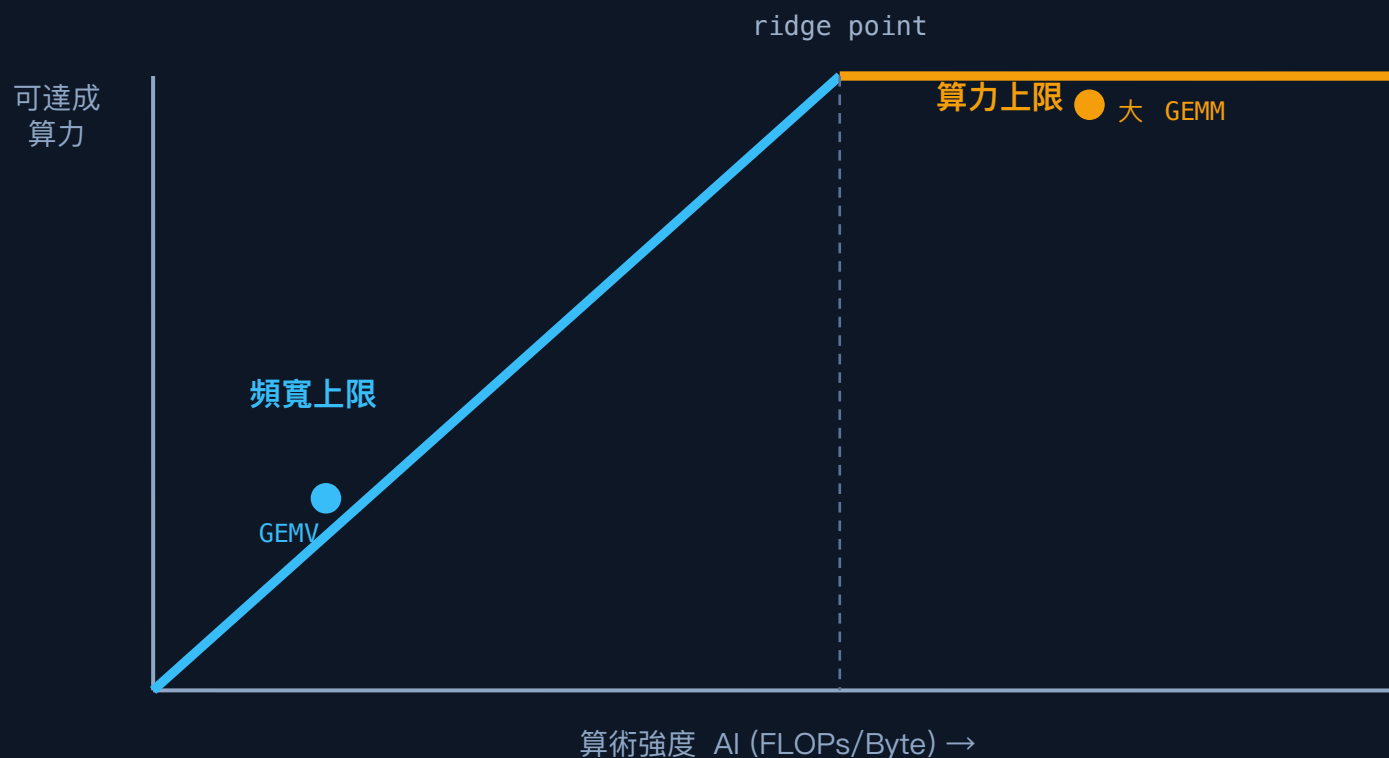
AI 高 → compute-bound

每搬一點資料就做很多運算。例：大方陣 GEMM，AI $\approx$ 數百。

AI 低 → memory-bound

大多時間在等資料。例：矩陣 $\times$ 向量 GEMV，AI $\approx$ 1—2 (fp16)。

10 Roofline：一眼看出你的天花板



- 斜線 = 頻寬上限
- (memory-bound)
- 平頂 = 算力上限
- (compute-bound)
- 轉折 = ridge point
- 點落在斜線上 → 受頻寬限，換更強算力也沒用

11 Ridge point 範例： H100

990

TFL0PS

BF16 tensor 峰值算力

3.35

TB/s

HBM3 記憶體頻寬

≈300

FL0Ps/Byte

ridge point = 算力 ÷ 頻寬

算術強度 < 300 的運算，再強的 tensor core 也餵不飽——你只能拿到「頻寬的速度」。

(數字為約略值，以官方規格為準)

12 990 TFLOPS 哪來的？時脈 × 單元數 × 每 cycle 運算數

$$\text{峰值算力} = \text{單元數} \times \text{每 cycle 運算數} \times \text{時脈 (clock)}$$

同一顆 H100	單元數	每 cycle 運算數	時脈	= 峰值
CUDA core 路徑 (fp32)	132 SM × 128	× 2 (FMA: $a \times b + c$ 一次算 2 FLOPs)	~2.0 GHz	≈ 67 TFLOPS
Tensor core 路徑 (bf16)	132 SM × 4	× ~1024 (小矩陣乘加一口氣做完)	~1.8 GHz	≈ 990 TFLOPS

頻寬是同一條公式：HBM3 ≈ 5120-bit 介面 × ~5.2 Gb/s/pin ÷ 8 ≈ 3.35 TB/s —— 「算力與頻寬都是：寬 × 時脈」。

兩條路徑時脈幾乎一樣、峰值差 ~15x—— 算力來自「每 cycle 運算數」不是 clock；GPU 時脈甚至比 CPU 低（Part 1）。

13 tiling：把資料留在晶片內重複用 → AI 變高

naive

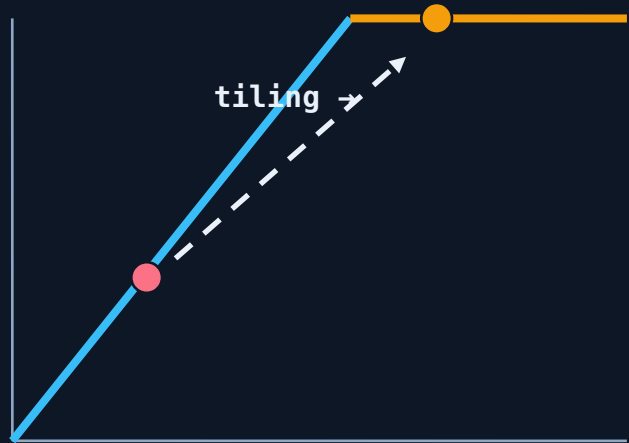
每個輸出都從 HBM 重讀整行整列 →
HBM 讀 $\sim O(N^3)$ 。

HBM 讀取：大

tiled

block 載入 shared memory 一次、晶
片內重複用 T 次 → HBM 讀 $\div T$ 。

小



同樣 FLOPs、更少 bytes
→ 推進 compute-bound

這就是 cuBLAS / tensor core kernel 都重度 tiling 的原因——Part 5 的 FlashAttention 是同一招的演算法版。

14 Training：吃算力，但更怕「裝不下」

為什麼通常 compute-bound

- 大 batch $\rightarrow$ 大 GEMM $\rightarrow$ AI 高
- forward / backward 都是大矩陣乘法
- 算力利用率高、餵得飽 tensor core

但記憶體同時要放下

- 權重 weights
- 啟動值 activations（留著等 backward）
- 梯度 gradients
- 優化器狀態（Adam $\approx$ 2x 參數，fp32）

訓練的痛點常是「裝不下」（容量）不是「餵不飽」（頻寬）—— 所以有梯度檢查點、ZeRO、offload。

15 Inference 兩階段: prefill vs decode

時間軸 →



Prefill — compute-bound

整段 prompt 一起算 → 大 GEMM、AI 高，像訓練的 forward。

Decode — memory-bound

一次一 token、batch 小 → GEMV、AI ≈ 1 。下一頁解謎題①。

16 謎題①解答： decode 每 token 重讀整份權重

每產生 1 個 token，就要把「整份權重」從 HBM 讀一遍。

14

GB

7B 模型 fp16 權重

4.2

ms / token

$14 \text{ GB} \div 3.35 \text{ TB/s}$

~240

tokens/s

batch=1 的延遲上限

「<5% 算力利用率」的根源：時間都花在搬權重，tensor core 在閒置。換更強算力救不了。

17 KV cache：省了重算，換來頻寬與容量壓力

是什麼

快取過去 token 的 Key / Value，讓每一步不必對整段序列重算 attention——是讓 decode 還能用的關鍵優化。

代價

- 隨序列長度線性增長
- 每步都要讀它 → 頻寬壓力
- 要存住它 → 容量壓力（限制 batch / context）

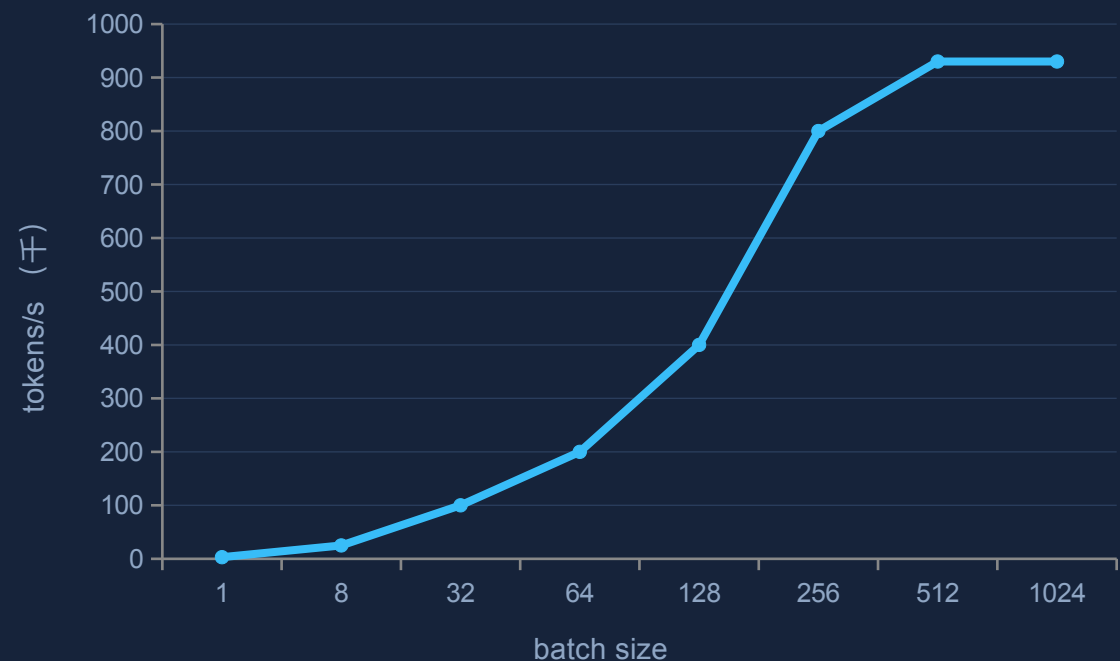
$$\text{KV 大小} \approx 2 \times \text{layers} \times \text{heads} \times \text{head_dim} \times \text{seq_len} \times \text{batch} \times \text{dtype}$$

context 越長 decode 每步要讀的越多 → 越 memory-bound。Part 5 的 GQA / Mamba 都是衝著它來的。

18 Batch 的魔法：throughput vs latency

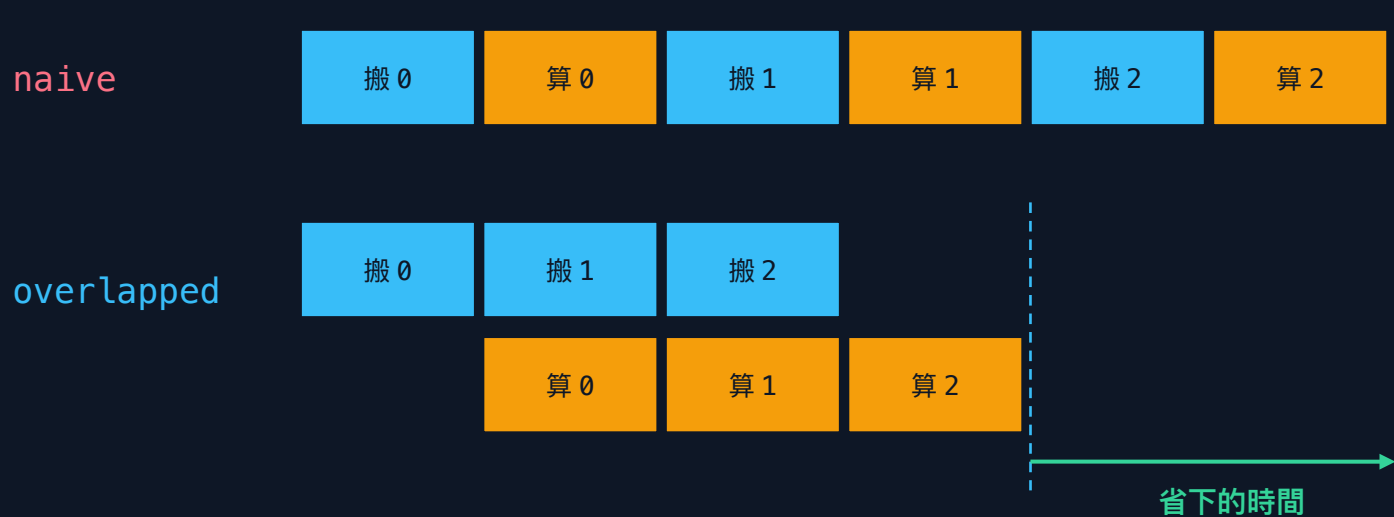
為什麼加大 batch 有用

- batch=1：讀一次權重只服務一個請求 → 浪費
- batch=N：同一次權重讀取服務 N 個請求
- → 吞吐近乎線性升、單步延遲幾乎不變
- 直到變 compute-bound 或 KV cache 裝不下



吞吐升、但「單一請求的延遲」沒變快——batch 是攤平權重讀取，不是加速。

19 壓軸： prefetch / overlap —— 把搬運藏在運算後面



第二條 stream 預取下一批：compute 一邊算、copy 一邊搬。DataLoader 的 `num_workers + pin_memory + prefetch` 是同一招。

搬運省不掉時就把它藏起來，讓昂貴的算力不再空等——資料搬遷篇的收束。

20 平行度不是硬體給的，是模型「暴露」出來的

batch

樣本之間天然獨立

pixel / 空間

卷積對每個位置獨立

channel / head

特徵維度切開算

token / 時間步

RNN：序列相依 ✕ attention：獨立 ✓

layer (pipeline)

層間相依，只能流水線

模型設計 = 決定把計算依賴圖畫成「深的鏈」還是「寬的樹」。RNN 暴露 $B \times H$ ，attention 暴露 $B \times T \times H$ 。

21 硬體 → 模型：CNN 等了 23 年的不是想法

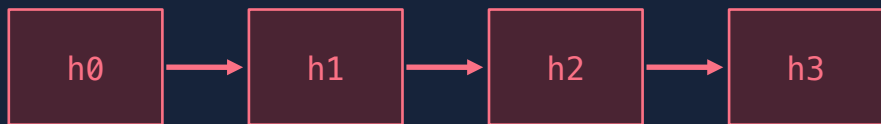


為什麼 CNN 跟 GPU 一拍即合：捲積經 im2col 攤平就是大 GEMM；pixel / channel / batch 全平行；權重重複用 → AI 高 → compute-bound。

Hardware lottery (Sara Hooker)：不是最好的想法贏，是最合當代硬體的想法贏。

22 硬體 → 模型：Transformer 為平行而生

RNN 訓練：T 步的鏈



h_t 依賴 h_{t-1} → token 軸只能一步步來；暴露平行度 $B \times H$ 。

Attention：整段一次 matmul



整段序列攤成 $(B \cdot T) \times H$ 的大 GEMM → token 軸從鏈變寬；暴露 $B \times T \times H$ 。

論文自己說的（2017）：RNN 的序列本質 “precludes parallelization”；Transformer “allowing significantly more parallelization”。

Transformer 贏不是因為 FLOPs 少（ $O(T^2)$ 其實更多），是把計算重排成 GPU 吃得下的形狀。



互動環節 ②

講者：切出去打開玩具級 Transformer 地圖（ $T=5$ 、 $d=6$ 、2 heads——同構縮小版，含 encoder↔decoder 全景與單一 matmul 的 L2/HBM 搬運）

[interactive/transformer_map.html](#)



三種執行模式切著看：訓練 / Prefill / Decode ——資料流、KV cache、GEMM→GEMV、利用率全部跟著變

建議動線：全景（encoder→memory M→decoder cross）→ Block → Attention（分叉=平行軸）→ Head 看矩陣 → 計算子：一個 matmul 的 L2↔HBM 搬運 → 硬體層 GPU/TPU/Groq；全程配 T/P/D 看 compute-bound→memory-bound、KV cache 串流 → 回來接下一頁

23 模型 → 硬體：箭頭反過來

Tensor core (Volta, 2017)

DL 負載九成是 GEMM → 做專用矩陣乘加單元

TPU systolic array

整顆晶片就是一台矩陣乘法機

H100 「Transformer Engine」

硬體功能直接用模型命名；fp8 為 transformer 訓練而做

H200：HBM 加到 141 GB

不是算力不夠，是 KV cache (Part 3) 吃容量 + 頻寬

精度 fp32→fp16→bf16→fp8→fp4

模型端證明訓得動 ⇔ 硬體端做出來，一步步往返

這是一個迴圈，不是單向因果：硬體決定哪些模型贏，贏的模型再回頭改造硬體。

24 TPU：另一條路 — systolic array（脈動陣列）

設計哲學：GEMM 專用到極致

- MXU = 128×128 個 MAC 排成陣列
- weight-stationary：權重釘在格子裡，資料流過去
- 沒有 warp、沒有動態排程——資料跟著 clock 齊步走
- XLA 編譯器把整個計算圖先排好（靜態）
- 代價：動態形狀 / 稀疏 / 分支不友善

Transformer 在 TPU 上

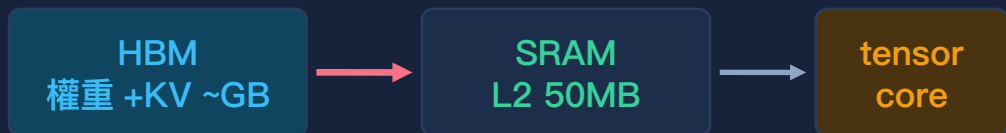
- 訓練 / prefill 的大 GEMM = 甜蜜點，利用率高
- decode 的小 GEMV 一樣餵不滿陣列 → 一樣 memory-bound
- bf16 就是 TPU 帶進主流的（模型→硬體→模型的迴圈）
- 也配 HBM——頻寬牆跨硬體成立

GPU 用「上萬 lane + 動態排程」、TPU 用「固定資料流 + 編譯器」——兩條路都是為 GEMM 而生；瓶頸物理相同。下一頁看第三條路 Groq。

25 Groq LPU：KV 塞不下晶片 → 乾脆把記憶體全做成 SRAM

GPU / TPU 的 decode 痛點

每步重串流



- KV cache 可達 GB $\gg$ L2 50MB $\gg$ shared 228KB
- → 放不進晶片 → 每 token 從 HBM 重新串流
- decode 卡在 3.35 TB/s 的 HBM 頻寬

Groq 的答案：砍掉 HBM、全用 SRAM



~230 MB SRAM / chip $\times$ 幾十~幾百顆

- SRAM ~80 TB/s $\approx$ HBM 的 20 $\times$ + → 沒有那條串流
- 一樣 memory-bound，但「記憶體」快 20 $\times$ + → decode token/s 大幅領先
- 代價：一顆只 230MB → 切片散到很多顆 + 確定性網路

記憶體牆是物理：GPU 在 HBM 上想辦法（FlashAttention 省 S、GQA 省 KV）、Groq 直接換掉 HBM——三條路都在繞同一道牆。互動地圖硬體層可切 GPU/TPU/Groq。

26 Case study CNN : FLOPs $\neq$ 速度

MobileNet : depthwise separable

- 紙上 FLOPs $\div 8-9$ ($9C^2 \rightarrow 9C + C^2$)
- 但 depthwise 每 byte 只做幾次運算 $\rightarrow$ AI 個位數、memory-bound
- channel 軸被拆散 $\rightarrow$ 餵不滿 tensor core
- 它的目標硬體本來就是手機 CPU

ConvNeXt (2022) : 反向操作

- 用 transformer 時代配方重設計 ResNet
- 敢用 7×7 大 kernel : FLOPs 多但平行度好、AI 高
- GPU 上「多而齊」常勝過「少而碎」

紙上 FLOPs 是 CPU 思維；GPU 上要問 bytes 與平行度——和 decode memory-bound 同一個道理。

27

Case study Transformer：演算法遷就記憶體階層

FlashAttention (2022)

- 數學完全不變 (exact attention)
- tiling + 線上 softmax $\rightarrow T \times T$ 矩陣從不落地 HBM (Part 2 的招)
- 同樣 FLOPs、bytes 大減 $\rightarrow$ 快 2-4x、記憶體 $O(T^2) \rightarrow O(T)$
- 改的不是數學，是資料的「住址」

MQA / GQA

- Part 3：decode 每 token 要搬整份 KV cache
- 乾脆改模型：多個 Q head 共用一組 KV head
- Llama 2 70B：64 Q head 共用 8 組 KV $\rightarrow$ KV bytes $\div 8$
- 為了頻寬連架構都改，接受輕微品質代價

28 Case study 混合架構：一個模型、兩種硬體人格

Mamba / SSM (2023)：雙目標共同設計

訓練時：parallel scan 攤平 T 軸 → 平行、compute-bound、吃滿 GPU | 推論時：退回遞迴、每 token $O(1)$ 狀態 → 沒有越長越肥的 KV cache

但固定狀態 = 有損記憶，精確長程檢索不如 attention → 2024 起主流是混血：

Jamba / Griffin (attention × SSM 交錯)；Conformer = 卷積 (局部) + attention (全局)，也是同類混血思路。

架構設計越來越像：在硬體約束下解最佳化問題。

29 設計 / 選模型前，先問三個硬體問題

①

平行度在哪个軸？有多寬？

餵不餵得飽上萬條 lane (B / T / pixel / channel / head)

②

每搬一個 byte 做幾次運算？

AI 落在 roofline 哪一區 (Part 2 的尺)

③

序列相依的鏈有多長？

Amdahl 的 $(1-p)$ ：訓練看 T 軸、推論看 decode 步數

用三問掃模型史： CNN (✓✓✓) → RNN (✗鏈長) → Transformer (訓練✓✓✓；decode ✗鏈長✗AI 低) → Flash/GQA (修 bytes)
→ Mamba/ 混合 (修鏈)

30 動手做：五個可重現的 demo

demo	對應篇章	展示概念	關鍵觀察
01_roofline_mini	Part 2	compute vs memory bound	瘦長矩陣掉進 memory-bound 區
02_pinned_vs_pageable	Part 4	進出站 / DMA	pinned H2D 快 $\sim 2\times$
03_decode_memory_bound	Part 3	decode 瓶頸 / batch 攤平	step 延遲平、tokens/s 線性升
04_prefetch_overlap	Part 4	把搬運藏在運算後面	overlap 提升 $\sim 1.7\times$ (示意)
05_flops_vs_parallelism	Part 5	FLOPs $\neq$ 速度	LSTM 輸給 FLOPs 多 $1.75\times$ 的 transformer ; depthwise $\div 8.7$ FLOPs 只 $\div 3.7$ 時間 (M2 實測)

程式在 repo 的 demos/ 下，每個資料夾一個 run.py + README。計時規範：warmup $\rightarrow$ 同步圍住 $\rightarrow$ 取中位數。

全系列帶走三句話

1

先問 `compute-bound` 還是 `memory-bound`——用算術強度那把尺。

很多「慢」是頻寬問題不是算力問題；換更貴的算力救不了 `memory-bound`。

2

瓶頸＝資料必經的最慢那段路；省不掉就藏起來。

tiling 把資料留在快的地方、pinned+overlap 把搬運藏在運算後面、選型看頻寬 + 容量。

3

模型與硬體互相塑造——看新架構先問三個硬體問題。

平行軸？每 byte 幾次運算？序列鏈多長？CNN → Transformer → 混合，就是輪流補洞。

速度的故事，大半是「資料在哪、怎麼搬、誰能平行」的故事。🎉